

i want you to help me create a plan to build a todo application using Next.js and SQLite

A good way to approach this is to build it in phases, starting with a solid foundation before adding more advanced features. Since you're using **Next.js** and **SQLite**, you'll have a fast, lightweight application that is easy to develop and deploy.

Project Overview

Project Name: NextDo (working title)

Tech Stack

Technology	Purpose
Next.js (App Router)	Frontend & Backend
TypeScript	Type safety
SQLite	Database
Prisma ORM	Database access
Tailwind CSS	Styling
shadcn/ui	UI Components
React Hook Form	Forms
Zod	Validation
Zustand (optional)	Client state management
date-fns	Date handling

Phase 1 – Project Setup

Objectives

Build the project foundation.

Tasks

- Create Next.js project
- Configure TypeScript
- Install Tailwind
- Install Prisma
- Configure SQLite
- Create initial Prisma schema
- Generate Prisma Client
- Configure ESLint & Prettier
- Create folder structure

Example structure

```
/app
  /(dashboard)
  /api
/components
/lib
/prisma
/public
/types
/utils
```

Deliverables

- Project runs
- Database connected
- Prisma working

Phase 2 – Database Design

Tables

Todo

```
id
title
description
completed
priority
dueDate
categoryId
createdAt
updatedAt
```

Category

```
id
name
color
createdAt
```

Tag (optional)

```
id
name
```

TodoTag

Many-to-many relationship.

Prisma Example

```
prisma
```

```

model Todo {
  id          Int          @id @default(autoincrement())
  title       String
  description  String?
  completed   Boolean      @default(false)
  priority    Priority      @default(MEDIUM)
  dueDate     DateTime?
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt

  categoryId  Int?
  category    Category? @relation(fields: [categoryId], references: [id])
}

model Category {
  id    Int    @id @default(autoincrement())
  name  String
  color String
  todos Todo[]
}

enum Priority {
  LOW
  MEDIUM
  HIGH
}

```

Phase 3 – Backend

API Routes

Todos

```

GET    /api/todos
POST   /api/todos
GET    /api/todos/:id
PATCH /api/todos/:id
DELETE /api/todos/:id

```

Categories

GET
POST
PATCH
DELETE

Validation

Use Zod.

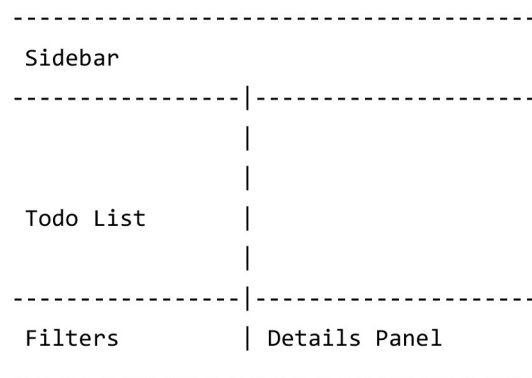
Example

TypeScript

```
const TodoSchema = z.object({  
  title: z.string().min(3),  
  description: z.string().optional(),  
  dueDate: z.date().optional(),  
  priority: z.enum(["LOW", "MEDIUM", "HIGH"])  
});
```

Phase 4 – UI

Layout



Components

Sidebar

- Categories

- Counts
- Settings

Header

- Search
- New Todo button
- Dark mode

Todo List

Each item shows

- checkbox
- title
- due date
- priority
- category color
- completed state

Details Panel

Displays

- description
- due date
- notes
- edit button

Phase 5 – CRUD

User should be able to

- Create task
- Edit task
- Delete task
- Complete task
- Undo completion

Phase 6 – Search & Filters

Search by

- title
- description

Filter by

- completed
- pending
- priority
- category
- due today
- overdue

Sort by

- newest
- oldest
- due date
- alphabetical
- priority

Phase 7 – Dashboard

Cards

Pending
Completed
Overdue
Due Today

Charts (optional)

- Completion rate
- Tasks per category
- Weekly productivity

Phase 8 – Nice UX

Animations

- Add task
- Delete task

- Complete task

Libraries

- Framer Motion
- Sonner (toast)

Phase 9 – Settings

Theme

- Light
- Dark
- System

Default

- Sort order
- Default priority
- Default category

Phase 10 – Testing

Unit tests

- API
- Validation
- Components

Libraries

- Vitest
- React Testing Library

Phase 11 – Deployment

Options

- Vercel (note: SQLite is best suited for local development or a single-instance deployment)
- Docker container
- VPS

Future Features

Recurring Tasks

Daily

Weekly

Monthly

Reminders

- Browser notifications

Calendar View

Month

Week

Agenda

Kanban Board

Todo

In Progress

Done

Drag and Drop

Move tasks between columns.

Attachments

Upload

- PDFs
- Images

Notes

Markdown support.

Labels

Multiple labels per task.

Statistics

- Productivity streak
- Average completion time
- Completion percentage
- Weekly report

Export

- CSV
- PDF
- JSON

Import

CSV

JSON

Suggested Development Timeline

Milestone

Goal

Week 1

Project setup, Prisma, SQLite, CRUD API

Milestone	Goal
Week 2	Todo list UI, create/edit/delete, categories
Week 3	Search, filters, sorting, responsive layout
Week 4	Dashboard, polish, animations, testing
Week 5	Optional features (calendar, recurring tasks, notifications)

Recommended Folder Structure

```
app/  
├── (dashboard)/  
│   ├── layout.tsx  
│   ├── page.tsx  
│   ├── todos/  
│   └── settings/  
├── api/  
│   ├── todos/  
│   └── categories/  
└── globals.css
```

```
components/  
├── layout/  
├── todos/  
├── dashboard/  
├── forms/  
└── ui/
```

```
lib/  
├── prisma.ts  
├── validation.ts  
├── actions/  
└── utils.ts
```

```
prisma/  
├── schema.prisma  
└── migrations/
```

```
hooks/  
types/
```

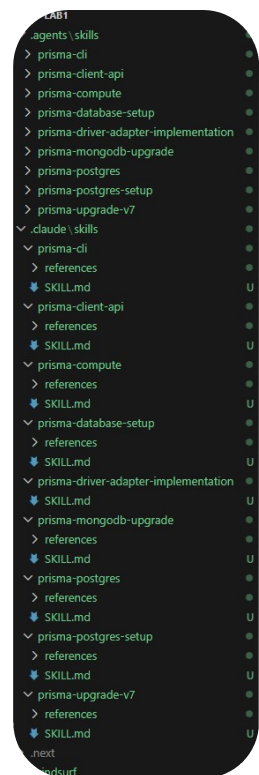
Development Principles

- Build vertical slices: implement one complete feature (database → API → UI) before moving to the next.

- Use **Server Components** for data fetching where appropriate and **Server Actions** or Route Handlers for mutations to keep the architecture simple.
- Validate all incoming data with Zod before writing to the database.
- Keep Prisma access centralized in a single `lib/prisma.ts` client.
- Design reusable UI components (buttons, dialogs, forms, task cards) from the beginning to make future features easier to add.

By following this roadmap, you'll end up with a clean, maintainable todo application that starts as a straightforward CRUD app and can naturally evolve into a feature-rich productivity tool without requiring major architectural changes.





These are the contents of the folders

This is actually very revealing.

The folders are **not being created by Prisma**. They are being created by an AI extension that provides "skills" for Prisma.

A few clues:

- `.claude/skills/`
- `.agents/skills/`
- `SKILL.md`
- `references/`
- Skills named:
 - `prisma-cli`
 - `prisma-client-api`
 - `prisma-compute`
 - `prisma-postgres-setup`

- `prisma-upgrade-v7`

These are **AI skill packs**, not Prisma files. Prisma has no concept of `SKILL.md` or a `skills` directory.

The biggest clue

I also noticed this in your screenshot:

```
.windsurf
```

That folder is created by **Windsurf** (formerly Codeium's AI editor/extension).

Even if you aren't actively using Windsurf, one of these is likely true:

- You previously opened this project in Windsurf.
- You have the Windsurf VS Code extension installed.
- You have the Codeium/Windsurf extension installed in VS Code.
- The project was cloned from somewhere that already contained these folders.

Let's confirm

In VS Code, press:

```
Ctrl + Shift + X
```

Search for:

- Windsurf
- Codeium

or run:

```
Bash
```

```
code --list-extensions
```

Look for anything like:

```
windsurf.windsurf  
codeium.codeium
```

or similar.

Another possibility

Those folders may have been committed to Git already.

Run:

```
Bash
```

```
git status
```

If they appear as:

```
tracked
```

they were already in the repository.

If they appear as:

```
Untracked files:
```

```
.agents/
```

```
.claude/
```

then something on your machine generated them.

The timing with prisma migrate

It's likely coincidence.

When you ran:

```
Bash
```

```
npx prisma migrate dev
```

the AI extension detected:

```
"A Prisma project has been initialized."
```

and automatically downloaded its Prisma skill packs, which is why you now have:

```
.claude/
```

```
.agents/
```

with Prisma-related documentation.

Prisma itself never creates those directories.

I recommend checking one more thing

Could you run:

Bash

```
code --list-extensions
```

and paste the output here?

I'm particularly interested in seeing if there's a Codeium, Windsurf, Continue, Cline, or another AI extension installed. That will let us identify the exact extension responsible.



```
code --list-extensions
llvm-vs-code-extensions.vscode-clangd
miramac.vscode-exec-node
ms-vscode-remote.remote-wsl
ms-vscode.cmake-tools
ms-vscode.cpp-devtools
ms-vscode.cpptools
ms-vscode.cpptools-extension-pack
ms-vscode.cpptools-themes
ms-vscode.makefile-tools
openai.chatgpt
Show more ▾
```

Thanks—that's interesting. Based on that list:

- ❌ No Windsurf extension
- ❌ No Codeium
- ❌ No Claude extension
- ❌ No Cline/Roo Code/Continue
- ✅ Only the official OpenAI ChatGPT extension (`openai.chatgpt`)

That means the folders are **not** coming from any currently installed VS Code extension.

However, your screenshot still shows a `.windsurf` directory in the project root. That strongly suggests one of these scenarios:

1. The project was created or opened in Windsurf previously, leaving behind its configuration.
2. Those folders are already part of the repository (perhaps from a template or another developer).
3. Another tool generated them before you removed/uninstalled it.

Let's determine whether they're tracked by Git

Run:

Bash

```
git ls-files .agents .claude .windsurf
```

If you see output like:

```
.agents/skills/prisma-cli/SKILL.md
.claude/skills/prisma-cli/SKILL.md
.windsurf/settings.json
```

then those files are **already tracked in Git** and were not created by `prisma migrate`.

If it prints nothing, they're untracked and were generated locally.

Also check when they were created

On Windows PowerShell:

PowerShell

```
Get-ChildItem .agents | Select Name, CreationTime
Get-ChildItem .claude | Select Name, CreationTime
```

If the creation time matches exactly when you ran `npx prisma migrate dev`, that tells us when they appeared.

One more thing I'd like you to check

The names of those skills are unusual:

- `prisma-compute`
- `prisma-driver-adapter-implementation`

- prisma-upgrade-v7

Those look like packaged AI documentation rather than configuration. Could you open one of these files and paste the first 20–30 lines?

For example:

```
.claude/skills/prisma-cli/SKILL.md
```

or

```
.agents/skills/prisma-cli/SKILL.md
```

That file will almost certainly identify the tool that generated the skills. It often contains a header like "Generated by...", a repository URL, or references to the agent framework that created it.

At this point, I'm confident of one thing: **Prisma itself does not generate** `.agents` , `.claude` , `.windsurf` , **or** `SKILL.md` **files**. Those directories originate from an AI tooling ecosystem, and the `SKILL.md` contents should tell us exactly which one.



if i delete these files will it break any part of my program?

No—if your application is a standard Next.js + Prisma project, deleting those folders **will not break your application**.

The folders you've shown are:

```
.agents/  
.claude/  
.windsurf/
```

These are **not required** by:

-  Next.js
-  Prisma
-  SQLite
-  React
-  TypeScript

-  npm

They're typically used for AI tooling (agent configuration, skills, or project metadata).

Before deleting them

Check whether they're tracked by Git:

Bash

```
git status
```

If they're listed as **untracked**, you can safely delete them:

Bash

```
rm -rf .agents .claude .windsurf
```

or on Windows PowerShell:

PowerShell

```
Remove-Item -Recurse -Force .agents, .claude, .windsurf
```

Then run your app:

Bash

```
npm run dev
```

and test:

- The app starts
- Prisma queries work
- `npx prisma migrate dev` still works
- `npx prisma studio` still opens

Everything should continue working normally.

If they come back...

If those folders are recreated after you delete them, then **something on your machine is generating them**. At that point we'd investigate what's watching your project.

One thing still puzzles me

You have a `.windsurf` folder but no Windsurf extension installed. That makes me suspect the project may have been created from a template or previously opened in Windsurf.

Could you show me the output of:

Bash

```
dir -Force
```

(or `ls -la` if you're using Git Bash)

I'm particularly interested in whether you also have files like:

- `.cursor`
- `.continue`
- `.codeium`
- `.vscode`

That will help determine where the project originally came from.



ChatGPT can make mistakes. Check important info.

The model used was Chatgpt Web [GPT-5.5]
This document was printed directly from the AI chat.